

TP10 : calcul des distances

ZFAI

INTRODUCTION

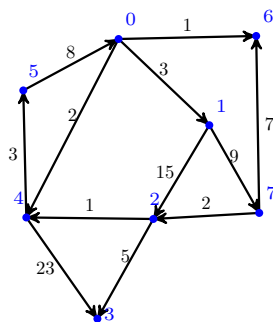
L'objectif de ce TP est de présenter et d'implémenter l'algorithme de Dijkstra, qui est un algorithme permettant de calculer les plus courts chemins dans le cas de graphes où les arcs ont des poids.

Algorithme de Dijkstra

Définition 1 (Graphe valué)

Un **graphe valué** G est un triplet (S, A, v) où S est un ensemble fini, A est une partie de $S \times S$ et $v : A \rightarrow \mathbb{R}^+$ est une application.

On représente en général un graphe par un dessin. Par exemple, le dessin



représente le graphe G_1 dont l'ensemble des sommets S est égal à $\{0, 1, 2, 3, 4, 5, 6, 7\}$, l'ensemble des arcs est donné par

$$\mathcal{A} = \left\{ \begin{array}{l} (0, 1), (0, 4), (0, 6), (1, 2), (1, 7), (2, 3), \\ (2, 4), (4, 3), (4, 5), (5, 0), (7, 2), (7, 6) \end{array} \right\}$$

et le poids d'un arc est indiqué sur l'arc. Par exemple, $v((0, 1)) = 3$.

un chemin dans un graphe est une suite finie de sommets $s_0, \dots, s_k$ où pour tout $i \in \{0, \dots, k-1\}$, le couple (s_i, s_{i+1}) est un arc. Le poids du chemin est la somme des poids des arcs constituant le chemin. Par exemple, le chemin $0 \rightarrow 1 \rightarrow 7 \rightarrow 6$ du graphe G_1 a comme poids $3 + 9 + 7 = 19$.

Un problème classique consiste à trouver les chemins de plus petit poids dans un graphe. On s'intéresse dans la suite à différentes versions de ce problème.

Dans la suite, les graphes étudiés sont des graphes dont l'ensemble des sommets est donné par un ensemble de la forme $\{0, 1, \dots, n-1\}$ avec $n \in \mathbb{N}$. Pour représenter un graphe valué en Python, on peut utiliser une liste d'adjacence. Par exemple, le graphe G_1 est représenté par la liste

$[[(1, 3), (4, 2), (6, 1)], [(2, 15), (7, 9)], [(4, 1), (3, 5)], [], [(3, 23), (5, 3)], [(0, 8)], [], [(2, 2), (6, 7)]]$.

On considère un graphe valué $G = (S, A, v)$ où v est une application à valeurs dans $\mathbb{R}^{+*}$.

L'algorithme de Dijkstra consiste à calculer les plus petites distances d'un sommet donné s vers les autres. Celui-ci est similaire à un algorithme de parcours : il suffit de choisir comme structure de données une file de priorité *FP*.

Cette structure correspond à un ensemble où chaque élément possède une certaine priorité qui est un réel. On rappelle les opérations présentes dans une telle structure :

- **est_vide**(FP) qui vérifie si la file de priorité est vide,
- **ajouter**(FP, p, x) qui ajoute à la file de priorité l'élément x avec la priorité p ,
- **extraire**(FP) qui récupère et supprime l'élément ayant la plus grande (ou plus petite suivant ce qu'on cherche) priorité.

Pour représenter une file de priorité, on utilise simplement une liste de couples Python de la forme (priorité, élément). Par exemple, la liste $[(1, 3), (4, 8), (2, 5)]$ représente une file de priorité à trois éléments, l'élément 3 a comme priorité 1, l'élément 8 a comme priorité 4, l'élément 5 a comme priorité 2.

On donne une présentation de l'algorithme de Dijkstra.

```

Dijkstra( $G, s$ )
/*  $G$  correspond au graphe, la fonction de poids est notée  $p$ ,  $s$  est le sommet de départ */
Initialiser un tableau  $d$  à l'infini;
 $d[s] \leftarrow 0$ ;
Initialiser une file  $FP$  contenant le sommet  $s$  avec comme valeur de priorité 0;
tant que  $FP$  est non vide faire
     $a, u := \text{extraire}(FP)$ ; /* On extrait l'élément ayant la priorité la plus petite */
    si  $a = d[u]$  alors
        /* un même élément  $u$  a pu être ajouté plusieurs fois avec des priorités différentes. On effectue un traitement qu'avec celui qui a la plus petite priorité */
        pour  $v$  voisin cible de  $u$  faire
            si  $d[v] > d[u] + p(u, v)$  alors
                /* un meilleur chemin a été trouvé. On met alors à jour les structures. */
                ajouter( $FP, d[u] + p(u, v), v$ );
                 $d[v] \leftarrow d[u] + p(u, v)$ ;
            fin
        fin
    fin
fin
retourner  $d$ 

```

Sur l'exemple G_1 avec le somme 0, déroulons l'application en précisant à chaque étape l'état de d , de FP et de l'élément extrait :

nombre d'itérations	d	FP	a, u
0	$[0, \infty, \infty, \infty, \infty, \infty, \infty, \infty]$	$[(0, 0)]$	$-$
1	$[0, 3, \infty, \infty, 2, \infty, 1, \infty]$	$[(1, 6), (3, 1), (2, 4)]$	0, 0
2	$[0, 3, \infty, \infty, 2, \infty, 1, \infty]$	$[(2, 4), (3, 1)]$	1, 6
3	$[0, 3, \infty, 25, 2, 5, 1, \infty]$	$[(3, 1), (5, 5), (25, 3)]$	2, 4
4	$[0, 3, 18, 25, 2, 5, 1, 12]$	$[(5, 5), (12, 7), (18, 2), (25, 3)]$	3, 1
5	$[0, 3, 18, 25, 2, 5, 1, 12]$	$[(12, 7), (18, 2), (25, 3)]$	5, 5
6	$[0, 3, 14, 25, 2, 5, 1, 12]$	$[(14, 2), (18, 2), (25, 3)]$	12, 7
7	$[0, 3, 14, 25, 2, 5, 1, 12]$	$[(18, 2), (19, 3), (25, 3)]$	14, 2
8	$[0, 3, 14, 19, 2, 5, 1, 12]$	$[(19, 3), (25, 3)]$	18, 2
9	$[0, 3, 14, 19, 2, 5, 1, 12]$	$[(25, 3)]$	19, 3
10	$[0, 3, 14, 19, 2, 5, 1, 12]$	$[\]$	19, 3

- Exercice 1.**
1. Le fichier `matrice.csv` contient en format texte une matrice d'adjacence d'un graphe à 285 sommets. Dans ce fichier, un coefficient à la ligne i et colonne j est égal à : -1 s'il n'y a pas d'arc direct de i à j , 0 s'il y a une transition instantanée, à un entier $n > 0$ qui correspond au nombre de minutes pour aller de i à j en passant par un arc direct. À l'aide de la fonction fournie dans le fichier `TP10.py`, stocker la matrice d'adjacence dans une variable (par exemple M).
 2. Écrire une fonction `conversion_matrice_liste(M)` prenant en argument une liste de listes d'entiers qui représente la matrice d'adjacence d'un graphe G valué et qui renvoie la liste d'adjacence du graphe G . Utiliser cette fonction pour calculer la liste d'adjacence de M .
 3. Écrire une fonction `dijkstra(G,s)` prenant en arguments une liste représentant un graphe G et un sommet s et qui renvoie la liste d calculée à l'aide de l'algorithme de Dijkstra.
 4. Le fichier `distances.csv` contient en format texte les différentes distances calculées avec l'algorithme de Dijkstra. Vérifier à l'aide d'un programme que vos résultats sont cohérents avec ceux du fichier. À noter : Pour la ligne i , la colonne j contient la distance optimale de i vers j .

Algorithme A^*

Dans un certain nombre de cas, on ne cherche pas la distance entre un sommet et tous les autres qui lui sont accessibles mais plutôt la distance entre un sommet source s et un sommet destination d . Il est possible d'aborder un tel problème à l'aide de l'algorithme A^* . Pour cela, on a besoin d'une fonction heuristique admissible.

On considère $G = (S, A, v)$ un graphe à poids positifs, s un sommet source, d un sommet destination. On note `distance_d` la fonction distance entre un sommet donné et d .

Dans ce contexte, on dit que H est une heuristique admissible, si pour tout $u \in S$, on a :

$$H(u) \leq \text{distance_d}[u]$$

Autrement dit, une fonction heuristique admissible ne surestime jamais la distance restante entre u et d .

Dans la suite, on présente l'algorithme A^* qui est une variante de l'algorithme de Dijkstra, où la façon de calculer les priorités est différente. On pourra remarquer que l'algorithme de Dijkstra correspond au cas où on considère comme heuristique admissible la fonction nulle.

```

A_etoile(G,s,d,H)
/* G correspond au graphe, la fonction de poids est notée p, s est le sommet de départ, d est le sommet de destination, H est
   une heuristique */
Initialiser un tableau distance à l'infini;
distance[s] ← 0;
Initialiser une file FP avec comme sommet initial s et avec la priorité distance[s] + H(s) tant que FP est non vide faire
    u := extraire(FP); /* On extrait l'élément ayant la priorité la plus faible */
    si u=d alors
        retourner distance[d]
    fin
    pour v voisin cible de u faire
        si distance[v] > distance[u] + p(u,v) alors
            ajouter v dans FP avec comme priorité distance[u] + p(u,v) + H(v) ou modifier la file FP en changeant la priorité de v par
                d[u] + p(u,v) + H(v);
            distance[v] ← distance[u] + p(u,v);
        fin
    fin
fin
retourner "pas de chemin"

```

- Exercice 2.**
1. Écrire une fonction **astar**(G,s,d,H) qui est une traduction de l'algorithme **astar**. L'argument H pourra être une liste de listes, où $H[u][v]$ est le temps estimé pour aller de u à v .
 2. La matrice d'adjacence donnée dans le fichier `matrice.csv` a été construite suivant un réseau ferré. Le fichier `localisation.csv` correspond à la position des différentes gares avec en première colonne la latitude, la seconde la longitude.
 - (a) Construire une liste de listes qui contient les positions données par le fichier `localisation.csv`.
 - (b) En déduire une fonction **temps_estime**(pos1,pos2) prenant en arguments les coordonnées géographiques de deux positions et qui renvoie le temps estimé pour aller de la position 1 à la position 2. On pourra utiliser l'approximation suggérée dans le fichier `TP10.py`.
 - (c) Calculer la liste de listes H telle que pour tout sommet u et v , l'entier $H[u][v]$ est le temps estimé pour aller de u à v .
 - (d) Calculer les distances à l'aide de l'algorithme A^* et vérifier la cohérence avec les résultats précédents.